МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"

Інститут **ІКНІ**

Кафедра **ПЗ**

Звіт

До лабораторної роботи № 2
На тему: *"Документування етапів проектування та кодування програми"*
З дисципліни: *"Вступ до інженерії програмного забезпечення"*

Лектор:

ст. викл. каф. ПЗ
Левус Є.В.

Виконала:

ст. гр. ПЗ-16

Киселиця Д. А.

Прийняв:

асист. каф. ПЗ

Самбір А. А.

« ____ » _____ 2022 р.

∑= ____ _ _ _ _ _ _ _ _ _ _

Львів – 2022

Тема: Документування етапів проектування та кодування програми.

Мета: Навчитися документувати основні результати етапів проектування та кодування найпростіших програм.

Теоретичні відомості

Варіант 7:

1. 6) Що таке шаблони проектування? Яка мета використання патернів проектування?

   Шаблони проектування (патерн, Design Pattern, англ.) – це багато разів застосовувана архітектурна конструкція, яка надає рішення загальної проблеми проектування в рамках конкретного контексту. Патерн не є закінченим зразком проекту, який може бути прямо перетворений в код, це опис або зразок для того, як вирішити завдання таким чином, щоб це можна було використовувати в різних ситуаціях.

2. 26) Які правила до найменування констант? Навести три приклади

   Константи, Enum, #define і Macro:

   Назви мають бути великими літерами і розділятися ‘_’.

```
24ons tint A_GLOBAL_CONSTANT = 5;
#define NUMBER_OF_ELEMENTS 5
enum { STATE_ERR, STATE_OPEN, STATE_RUNNING,
STATE_DYING};
```

3. 34) Чим мова програмування С принципово відрізняється від мови програмування С++?

   Найбільша принципова відмінність між комп'ютерною мовою С, вперше випущеною в 1972 році, і мовою програмування С ++, випущеною в 1983 році, майже через 10 років, полягає в тому, що оригінальний С є прикладом процедурної мови обчислень, тоді як С ++ - це представлення принципів об'єктно-орієнтованого програмування (ООП). Як результат, С ++ підтримує класи та об'єкти, нові способи структурування коду та нові способи думати про програмування.

Постановка завдання

Частина I. У розробленій раніше програмі до лабораторної роботи з дисципліни «Основи програмування» внести зміни – привести її до модульної структури, де модуль – окрема функція-підпрограма. У якості таких функцій запрограмувати алгоритми зчитування та запису у файл, сортування, пошуку, редагування, видалення елементів та решта функцій згідно варіанту.

Частина II. Сформувати пакет документів до розробленої раніше власної програми:

1. схематичне зображення структур даних, які використовуються для збереження інформації ;
2. блок-схема алгоритмів – основної функції й двох окремих функцій-підпрограм (наприклад, сортування та редагування);
3. текст програми з коментарями та оформлений згідно вище наведених рекомендацій щодо забезпечення читабельності й зрозумілості. Для схематичного зображення структур даних, блок-схеми алгоритму використати редактор MS-Visio.

Частина III. У редакторі MS-Visio розробити зразки фігур, які були використані для схематичного зображення структур даних програм, як готові трафарети до використання . Сформувати свою бібліотеку фігур – окремий користувацький файл із використаними зразками.

Отримані результати

1. Схематичне зображення структур даних, які використовуються для збереження інформації (рис.1):

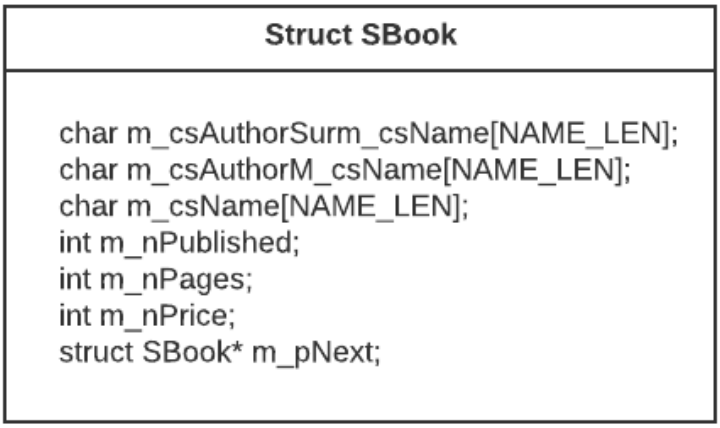

Рис. 1

2. Блок-схеми алгоритмів – основної функції та двох-підпрограм: main (рис.2), InsertSorted (рис.3), DeleteBooks (рис.4):

Початок
Оголошення змінних
Зчитування списку книг з файлу
Виведення початкового списку
Додавання нової книги у список
Виведення нового списку
Видалення книг, з кількістю сторінок менше 50
Виведення нового списку
Звільнення пам'яті
Кінець

Рис.2

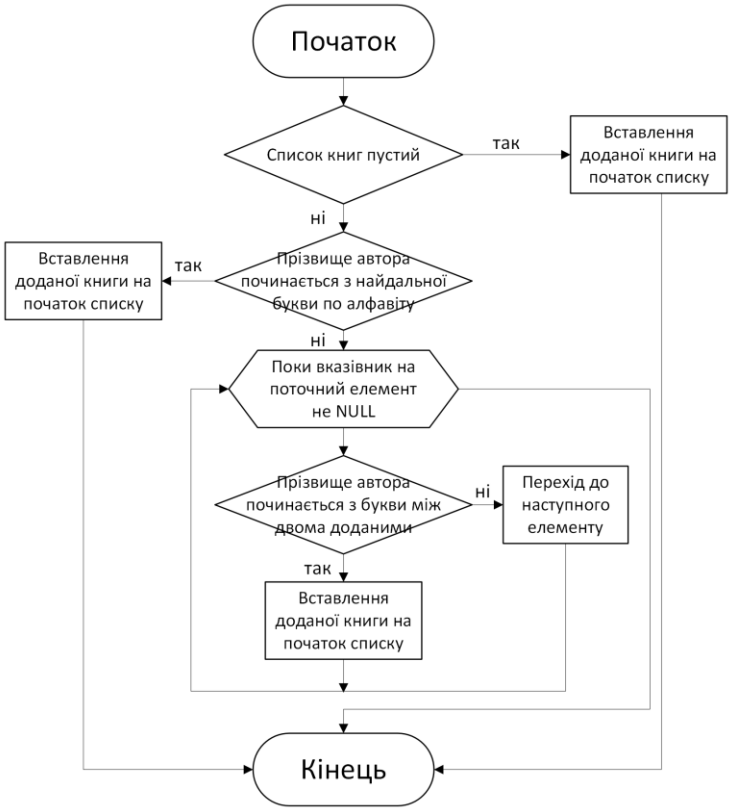
Початок
Список книг пустий
так
Вставлення доданої книги на початок списку
ні
Прізвище автора починається з найдальної букви по алфавіту
так
Вставлення доданої книги на початок списку
ні
Поки вказівник на поточний елемент не NULL
Прізвище автора починається з букви між двома доданими
ні
Перехід до наступного елементу
так
Вставлення доданої книги на початок списку
Кінець

Рис.3

Рис.4

3. Текст програми з коментарями та оформлений згідно з наведеними рекомендаціями щодо забезпечення читабельності й зрозумілості:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
// максимальна довжина для полів Автор та назви
#define NAME_LEN 50

/*Структура Книга, що містить наступні поля: прізвище і ім'я автора,
назву книги, рік видання, кількість сторінок та вартість.*/
typedef struct SBook
{
    char m_csAuthorSurm_csName[NAME_LEN];
    char m_csAuthorM_csName[NAME_LEN];
    char m_csName[NAME_LEN];
    int m_nPublished;
    int m_nPages;
    int m_nPrice;
    struct SBook* m_pNext;
} SBook;

//Фукнція приймає 2 вказівники на елементи списку
//Порівнює їх за першими літерами прізвищ авторів
//Повертає змінну типу int
int CompareAuthor(SBook* pa, SBook* pb)
{
    return (-strcmp(pa->m_csAuthorSurm_csName, pb->m_csAuthorSurm_csName));
}

//Фукнція приймає вказівник на елемент списку
//Порівнює за кількістю сторінок
```

```
//Повертає змінну типу int
int ComparePages(SBook* pa)
{
    if (pa->m_nPages < 50)
        return 1;
    else
        return 0;
}

//Функція приймає голову списку та вказівник на новий елемент списку.
//Вставляє книги у відсортований за протилежним алфавітним порядком список.
//Функція нічого не повертає
void InsertSorted(SBook** pHead, SBook* new_SBook)
{
    if (!(*pHead))
    {
        (*pHead) = new_SBook;
        (*pHead)->m_pNext = NULL;
    }
    else
    {
        if (CompareAuthor(*pHead, new_SBook) > 0)
        {
            (new_SBook)->m_pNext = *pHead;
            (*pHead) = new_SBook;
            return;
        }

        SBook* pCurr = *pHead;
        while (pCurr->m_pNext)
        {
            if (CompareAuthor(pCurr, new_SBook) < 0 && CompareAuthor(pCurr-
>m_pNext, new_SBook) >= 0)
            {
                new_SBook->m_pNext = pCurr->m_pNext;
                pCurr->m_pNext = new_SBook;
                return;
            }
            else
            {
                pCurr = pCurr->m_pNext;
            }
        }

        pCurr->m_pNext = new_SBook;
        new_SBook->m_pNext = NULL;
    }

}

//Функція приймає голову списку та вказівник на файл
// Функція зчитує інформацію про книги з файлу.
//Зчитана інформація про книги формується у відсортований однозв'язний список.
//Функція нічого не повертає
void GetBooksFromFile(SBook** pHead, FILE* fp)
{
    int nSymb = 0;
    while ((nSymb = getc(fp)) != EOF)
    {
        SBook* new_SBook = (SBook*)malloc(sizeof(SBook));

        fscanf(fp, "%s %s %s %i %i %i", new_SBook->m_csAuthorM_csName,
new_SBook->m_csAuthorSurm_csName, new_SBook->m_csName, &new_SBook->m_nPublished,
&new_SBook->m_nPages, &new_SBook->m_nPrice);

        InsertSorted(pHead, new_SBook);
    }

    fclose(fp);
}
```

```
//Функція приймає голову списку.
//Функція виводить список книг на екран у вигляді таблиці.
//Функція нічого не повертає.
void PrintData(SBook* pHead)
{
    SBook* pTemp = pHead;

    printf("|AUTHOR
|M_CSNAME\t\t\t\t|YEAR\t|M_NPAGES\t|M_NPRICE\n");
    while (pTemp)
    {
        printf("|%10s %10s\t|%25s\t|%5i\t|%5i\t|%5i\n", pTemp-
>m_csAuthorM_csName, pTemp->m_csAuthorSurm_csName, pTemp->m_csName, pTemp-
>m_nPublished, pTemp->m_nPages, pTemp->m_nPrice);
        pTemp = pTemp->m_pNext;
    }
}

/* Функція приймає голову списку.
Інформація про нову книгу зчитується з клавіатури.
Нова вставляється у відсортований у протилежному
алфавітному порядку список на необіхдну позицію.
Функція нічого не повертає.*/
void AddBook(SBook** pHead)
{
    printf("Enter SBook info: \n");
    SBook* add_SBook = (SBook*)malloc(sizeof(SBook));
    scanf("%s %s %s %i %i %i", add_SBook->m_csAuthorM_csName, add_SBook-
>m_csAuthorSurm_csName, add_SBook->m_csName, &add_SBook->m_nPublished,
&add_SBook->m_nPages, &add_SBook->m_nPrice);
    InsertSorted(pHead, add_SBook);
}

//Функція приймає голову списку.
//Функція видаляє зі списку книги з кількістю сторінок менше 50.
//Функція нічого не повертає.
void DeleteBooks(SBook** pHead)
{
    SBook* pCurr = *pHead;
    SBook* pPrev = NULL;
    while (pCurr)
    {
        if ((ComparePages(pCurr) == 1) && !pPrev)
        {
            (*pHead) = pCurr->m_pNext;
            free(pCurr);
            pCurr = (*pHead);
            continue;
        }

        if (ComparePages(pCurr))
        {
            pPrev->m_pNext = pCurr->m_pNext;
            free(pCurr);
            pCurr = pPrev->m_pNext;
            continue;
        }
        pPrev = pCurr;
        pCurr = pCurr->m_pNext;
    }
}

//Функція приймає голову списку.
//Функція видаляє список та звільняє пам’ять.
//Функція нічого не повертає.
void FreeMemory(SBook* pCurrent)
{
    if (pCurrent->m_pNext) FreeMemory(pCurrent->m_pNext);
    free(pCurrent);
}
```

```c
int main()
{
    char cChoice = '0';
    FILE* fp = fopen("Books.txt", "r");
    SBook* pHead = NULL;

    //отримую дані з файлу
    GetBooksFromFile(&pHead, fp);
    //вивід даних
    PrintData(pHead);

    printf("Add new Book / Delete matching Books (a/d)? : ");
    scanf("%c", &cChoice);
    if (cChoice == 'a')
    {
        // отримую дані та додаю нові книги у посортований список
        AddBook(&pHead);
    }
    else if (cChoice == 'd')
    {
        // видаляю дані про книги з кількістю сторінок менше 50
        DeleteBooks(&pHead);
    }

    printf("\n\t\t\t\tNEW DATA: \n");
    //вивід даних
    PrintData(pHead);

    //звільнення пам'яті
    FreeMemory(pHead);
    free(pHead);
    pHead = NULL;

    return 0;
}
```

Висновок: Під час виконання цієї лабораторної роботи я закріпила знання про документування основні результати проектування та кодування найпростіших програм. Сформувала пакет документів до розробленої раніше власної програми (схематичне зображення використаних структур даних (масив, список), блок-схеми алгоритмів (головної програми, функцій видалення та сортування елементів), код програми, який привела до модульної структури).